```
# Name : Aishwarya Laxmikant Lonikar
# CS AI - D
# Roll No. 2
# Subject : Machine Learning
# PRN : 12410541
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score, mean_squared_error

df = pd.read_csv('/content/sample_data/uber.csv')
print(df.shape)
df.head()
```

```
(26782, 9)
```

Unnamed: 0	key	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitu
	2015-05-07		2015-05-07				

```
df.dropna(inplace=True)
df.drop_duplicates(inplace=True)

df = df[(df['fare_amount'] > 0) & (df['fare_amount'] < 200)]

df = df[(df['passenger_count'] >= 1) & (df['passenger_count'] <= 6)]

df = df[
    (df['pickup_latitude'].between(-90, 90)) &
    (df['dropoff_latitude'].between(-90, 90)) &
    (df['pickup_longitude'].between(-180, 180)) &
    (df['dropoff_longitude'].between(-180, 180))
]

df.shape
```

```
(20262, 8)
```

```
def haversine(lat1, lon1, lat2, lon2):
    R = 6371

    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])

    dlat = lat2 - lat1
    dlon = lon2 - lon1

    a = np.sin(dlat/2)**2 + np.cos(lat1)*np.cos(lat2)*np.sin(dlon/2)**2
    c = 2 * np.arcsin(np.sqrt(a))

    return R * c

df['distance_km'] = haversine(
    df['pickup_latitude'],
    df['pickup_longitude'],
```

```
    df['dropoff_latitude'],
    df['dropoff_longitude']
)

df = df[(df['distance_km'] > 0) & (df['distance_km'] < 50)]

df.head()
```

	Unnamed: 0	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_c
0	24238194	7.5	1431028326000000000	-73.999817	40.738354	-73.999512	40.723217	
1	27835199	7.7	1247861096000000000	-73.994355	40.728225	-73.994710	40.750325	
2	44984355	12.9	1251150300000000000	-74.005043	40.740770	-73.962565	40.772647	
3	25894730	5.3	1246004541000000000	-73.976124	40.790844	-73.965316	40.803349	
8	15822268	9.7	1329471120000000000	-73.975187	40.745767	-74.002720	40.743537	

Next steps: New interactive sheet

```
def remove_outliers_iqr(data, cols):
    for col in cols:
        Q1 = data[col].quantile(0.25)
        Q3 = data[col].quantile(0.75)
        IQR = Q3 - Q1
        data = data[(data[col] >= (Q1 - 1.5*IQR)) & (data[col] <= (Q3 + 1.5*IQR))]
    return data

df = remove_outliers_iqr(df, ['fare_amount', 'distance_km'])
df.shape
```

```
(18842, 9)
```

```
numeric_df = df[[
    'fare_amount',
    'pickup_longitude',
    'pickup_latitude',
    'dropoff_longitude',
    'dropoff_latitude',
    'passenger_count',
    'distance_km'
]]

# Pearson
pearson_corr = numeric_df.corr(method='pearson')

# Spearman
spearman_corr = numeric_df.corr(method='spearman')

# Kendall
kendall_corr = numeric_df.corr(method='kendall')

print("Pearson Correlation:\n", pearson_corr)
print("\nSpearman Correlation:\n", spearman_corr)
print("\nKendall Correlation:\n", kendall_corr)
```

```
dropoff_latitude           0.665541          1.000000         -0.018189
passenger_count           -0.020786         -0.018189          1.000000
distance_km                0.011509         -0.030254          0.020220

                   distance_km
fare_amount           0.803634
pickup_longitude     -0.041311
pickup_latitude      -0.046744
dropoff_longitude     0.011509
dropoff_latitude     -0.030254
passenger_count       0.020220
distance_km           1.000000

Kendall Correlation:
                    fare_amount  pickup_longitude  pickup_latitude  \
fare_amount           1.000000         -0.041927        -0.046257
pickup_longitude     -0.041927          1.000000         0.509648
pickup_latitude      -0.046257          0.509648         1.000000
dropoff_longitude    -0.025470          0.332444         0.294312
dropoff_latitude     -0.052701          0.309042         0.437394
passenger_count       0.017011         -0.016811        -0.016239
distance_km           0.627191         -0.027487        -0.031347

                   dropoff_longitude  dropoff_latitude  passenger_count  \
fare_amount               -0.025470         -0.052701         0.017011
pickup_longitude           0.332444          0.309042        -0.016811
pickup_latitude            0.294312          0.437394        -0.016239
dropoff_longitude          1.000000          0.487657        -0.016662
dropoff_latitude           0.487657          1.000000        -0.014590
passenger_count           -0.016662         -0.014590         1.000000
distance_km                0.007663         -0.020469         0.016168

                   distance_km
fare_amount           0.627191
pickup_longitude     -0.027487
pickup_latitude      -0.031347
dropoff_longitude     0.007663
dropoff_latitude     -0.020469
passenger_count       0.016168
distance_km           1.000000
```

```
X = df[['distance_km', 'passenger_count']]
y = df['fare_amount']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
lr = LinearRegression()
ridge = Ridge(alpha=1.0)
lasso = Lasso(alpha=0.01)

lr.fit(X_train, y_train)
ridge.fit(X_train, y_train)
lasso.fit(X_train, y_train)
```

```
▾ Lasso
Lasso(alpha=0.01)
```

```
y_pred_lr = lr.predict(X_test)
y_pred_ridge = ridge.predict(X_test)
y_pred_lasso = lasso.predict(X_test)
```

```
def evaluate(name, y_true, y_pred):
    r2 = r2_score(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    print(name)
    print("R2 Score:", r2)
    print("RMSE:", rmse)
    print("-"*40)
    return r2, rmse

r2_lr, rmse_lr = evaluate("Linear Regression", y_test, y_pred_lr)
```

```
r2_ridge, rmse_ridge = evaluate("Ridge Regression", y_test, y_pred_ridge)
r2_lasso, rmse_lasso = evaluate("Lasso Regression", y_test, y_pred_lasso)
```

```
Linear Regression
R2 Score: 0.6162371931462034
RMSE: 2.050575500398166
----------------------------------------
Ridge Regression
R2 Score: 0.6162366566377726
RMSE: 2.0505769337713846
----------------------------------------
Lasso Regression
R2 Score: 0.616095993795255
RMSE: 2.05095270383811
----------------------------------------
```

```
results = pd.DataFrame({
    'Model': ['Linear Regression', 'Ridge Regression', 'Lasso Regression'],
    'R2 Score': [r2_lr, r2_ridge, r2_lasso],
    'RMSE': [rmse_lr, rmse_ridge, rmse_lasso]
})

results
```

	Model	R2 Score	RMSE
0	Linear Regression	0.616237	2.050576
1	Ridge Regression	0.616237	2.050577
2	Lasso Regression	0.616096	2.050953

Next steps: New interactive sheet